SECURELINK VPN LAB

Research Project | Secure VPN for Remote Access

VPN Architecture Design Document

Establishing a Secure VPN for Remote Access

Deliverable D1 of 6

Organisation	Research project (lab build)
Author	Kshitij Rajesh Randhire
Version	1.0 (final)
Basis	Lab build on VMware; evidence IDs [E01]-[E38] refer to the evidence screenshot set
Status	Lab design document. Not certified and not presented as production-ready.

Contents

1 Scope and objectives
2 Address plan
3 Topology diagrams
4 Technology selection
5 Data flow
6 Split-tunnel rationale
7 Lab-to-production mapping
8 Implementation status
9 Limitations
Appendix A Evidence cited

1 Scope and objectives

The brief asks for a VPN that gives a multinational corporation both site-to-site and client-to-site access, with confidentiality, integrity and authentication. This document describes what I actually built and tested in a small lab for this research project, and how it would carry over to a production design. The lab runs on my Windows 10 laptop under VMware (16 GB RAM) on a NAT network, 192.168.232.0/24, and all three virtual machines share that one host.

I label each item as Implemented and verified, Implemented but limited, or Recommended, not implemented. Evidence is cited as [E05] and so on; the screenshots are in the Evidence folder.

2 Address plan

Element	Address / network	Role	Evidence
gw-hq	192.168.232.131 (Ubuntu 24.04.5)	HQ gateway: OpenVPN 2.6.19, strongSwan, iptables, rsyslog sender, fail2ban	[E10]
gw-branch	192.168.232.132	Branch gateway: strongSwan, rsyslog receiver	[E01]
Kali client	192.168.232.128 (Kali 2026.1, OpenVPN 2.7.5)	Remote user device	[E05]
HQ LAN (simulated)	10.10.1.0/24, dummy0 10.10.1.1 on gw-hq	Stands in for corporate network; no real hosts	[E10]
Branch LAN (simulated)	10.10.2.0/24, dummy0 10.10.2.1 on gw-branch	Stands in for branch network; no real hosts	[E01]
OpenVPN instance 1	UDP 1194, tun0 10.8.0.1/24; client 10.8.0.2	Client-to-site pool 1	[E10][E16]
OpenVPN instance 2	UDP 1195, tun1 10.9.0.1/24; client 10.9.0.2	Client-to-site pool 2	[E10][E16]

3 Topology diagrams

3.1 Site-to-site

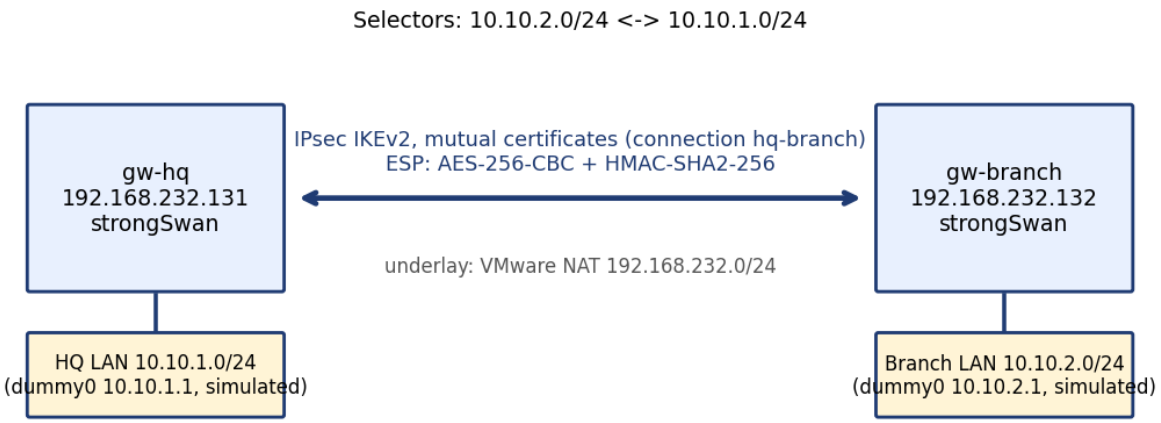


Figure 1. Site-to-site IPsec topology between gw-hq and gw-branch [E01][E10]

The tunnel uses strongSwan IKEv2 with mutual certificate authentication (connection hq-branch). IKE proposal: AES_CBC_256, PRF_HMAC_SHA2_256, HMAC_SHA2_256_128, MODP_2048. ESP: AES_CBC-256 with HMAC_SHA2_256_128 (CBC with HMAC, not GCM). Selectors are 10.10.2.0/24 <-> 10.10.1.0/24. A ping across the tunnel returned 4/4 with 0% loss and ESP packets were captured [E01].

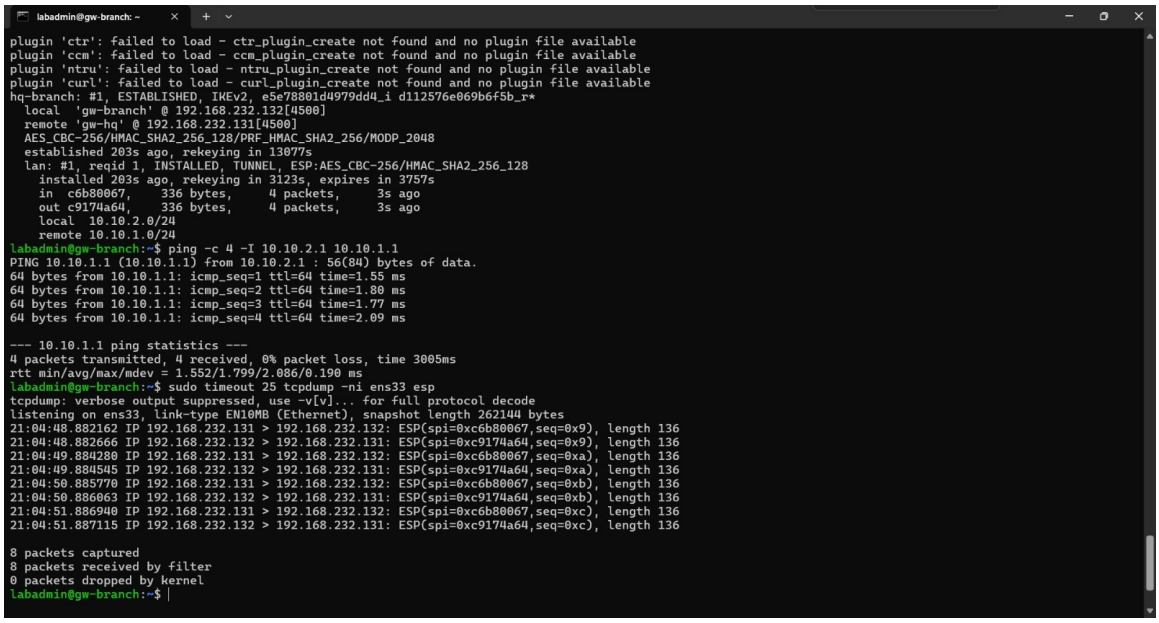


Figure 2. IPsec tunnel ESTABLISHED, ping 0% loss, ESP in tcpdump [E01]

3.2 Client-to-site

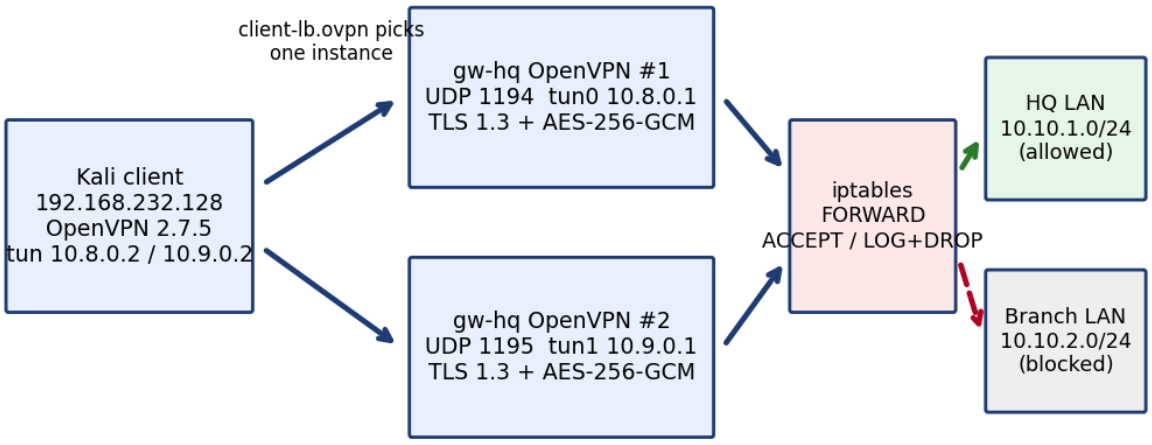


Figure 3. Client-to-site OpenVPN topology with two instances and iptables control [E05][E10][E11][E16]

Two OpenVPN instances run on gw-hq: UDP 1194 (tun0, 10.8.0.0/24) and UDP 1195 (tun1, 10.9.0.0/24) [E16]. Both negotiate TLS 1.3 with TLS_AES_256_GCM_SHA384, an RSA 2048 server certificate and X25519, with AES-256-GCM for data [E05]. The client authenticates with a certificate plus a TOTP code (see D2).

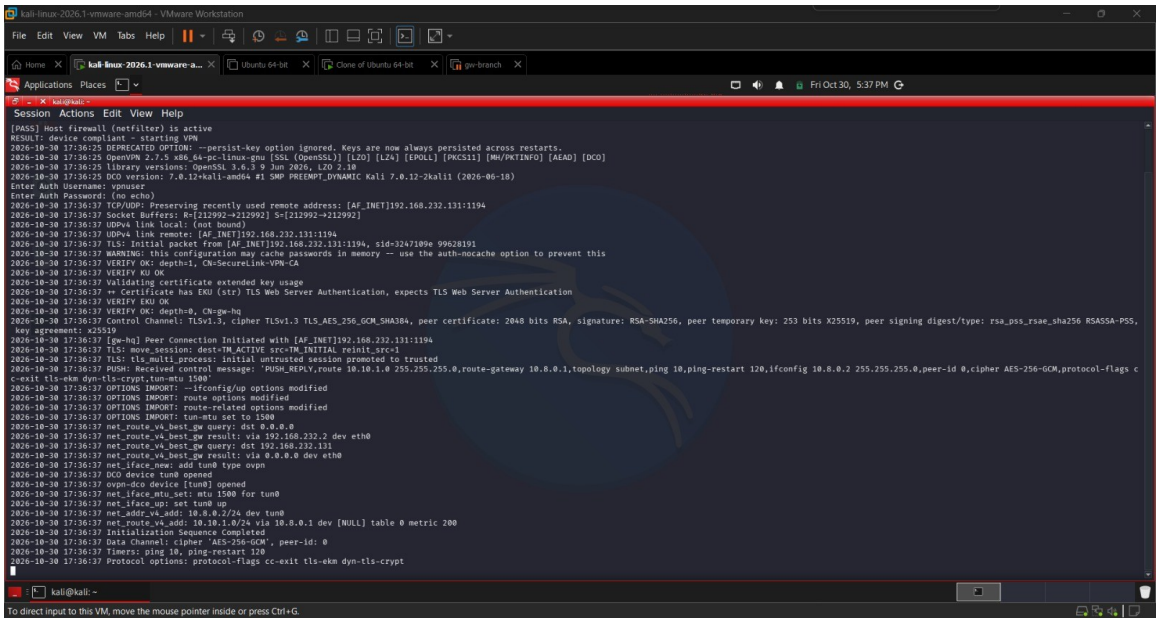


Figure 4. Compliant client connects; TLS 1.3, AES-256-GCM, one pushed route [E05]

4 Technology selection

I chose open-source tools for cost and for fit with the brief. I compared the alternatives on paper only; none of them was deployed or tested.

Need	Chosen (built)	Alternatives assessed on paper	Reason for choice

Site-to-site	strongSwan, IPsec IKEv2, certificates	WireGuard; OpenVPN site-to-site	IPsec is the standard for gateway-to-gateway and interoperates with most vendors
Client-to-site	OpenVPN (TLS), UDP	WireGuard; SoftEther	Mature TLS-based client with clients for Linux, Windows and macOS, and PAM plugin support for TOTP
Authentication	Certificates + TOTP via PAM (pam_google_authenticator)	FreeRADIUS	Fewer components for a small lab; RADIUS recommended for central user management at scale
Certificates	Easy-RSA / OpenSSL 3.0.13	Larger PKI tooling	Sufficient for a lab; two-tier PKI recommended
Logging	rsyslog, journald, fail2ban	ELK stack	Light enough for the lab hardware; ELK suggested for searching and dashboards
Monitoring	OpenVPN status file, logs	Nagios Core; Zabbix	Not deployed; see D4

Status: ELK, Nagios, Zabbix, FreeRADIUS, WireGuard and SoftEther are not part of the lab build.

5 Data flow

1. Client opens a UDP session to gw-hq (1194 or 1195) using the tls-crypt protected channel and a client certificate [E10].
2. gw-hq checks the client certificate against the CA and the CRL (crl-verify) and runs the PAM plugin, which validates the TOTP code typed at the password prompt [E07][E09].
3. A TLS 1.3 session is set up; the data channel uses AES-256-GCM with auth SHA256 [E05][E10].
4. gw-hq pushes the route 10.10.1.0/24 and assigns the tunnel address [E05][E07].
5. Traffic leaving the tunnel goes through iptables FORWARD: ACCEPT only to 10.10.1.0/24, otherwise LOG with prefix VPN-DENY then DROP [E11].
6. Events are logged locally and forwarded to gw-branch over TCP 514 (see D4) [E27][E28].

6 Split-tunnel rationale

The client profile ignores redirect-gateway and dhcp-option, so only 10.10.1.0/24 is routed through the VPN [E06]. Internet traffic stays on the user's own connection.

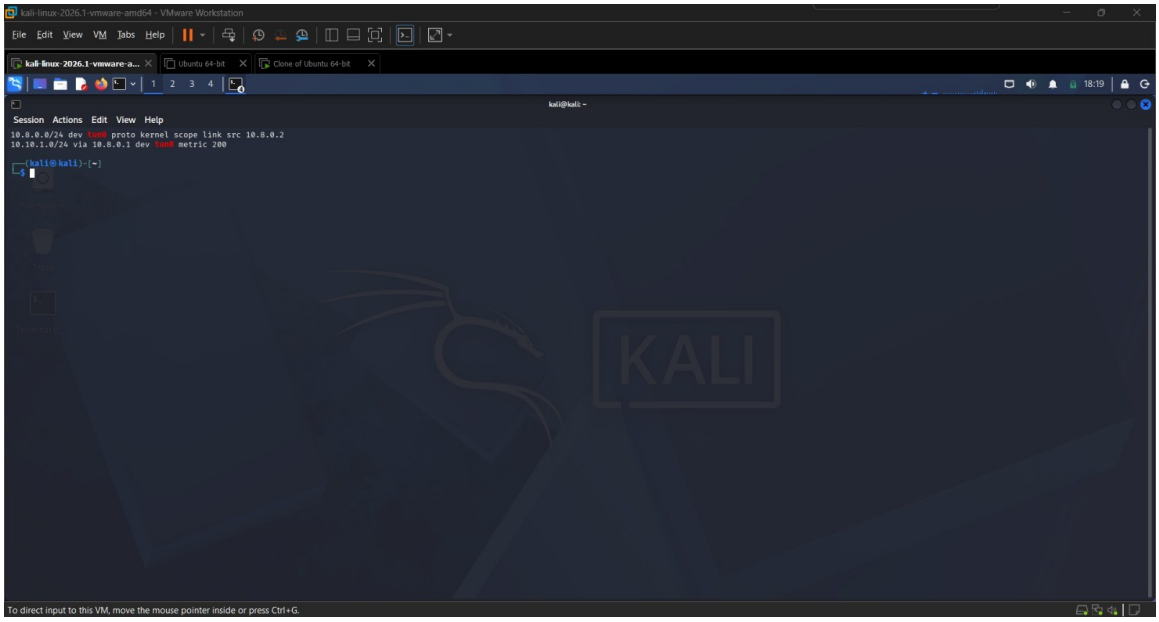


Figure 5. Client routing table with a single pushed route [E06]

Aspect	Split tunnel (built)	Full tunnel (not built)
Gateway load	Only corporate traffic	All traffic, higher load
Inspection of internet traffic	None by the company	Possible at the gateway
Exposure of client	Client is reachable from its local network while connected	Reduced, but depends on client firewall

I chose split tunnelling to keep the load low on a small lab gateway. A company with stricter requirements could move to a full tunnel, but I did not test that.

7 Lab-to-production mapping

Lab element	Production recommendation	Status
dummy0 simulated LANs	Real HQ and branch subnets with firewall rules per segment	Recommended, not implemented
Single self-signed CA	Two-tier PKI with an offline root	Recommended (planned, not done)
One client certificate / account	Per-user certificates and accounts	Recommended, not implemented
One gateway with two instances	Multiple gateways with real load balancing	Recommended, not implemented
TCP 514 without TLS	TLS-protected syslog	Recommended, not implemented
Windows/macOS clients	OpenVPN GUI (Windows); Tunnelblick or OpenVPN Connect (macOS)	Recommended, not tested

8 Implementation status

Category	Items
Implemented and verified	IPsec site-to-site tunnel [E01]; OpenVPN client-to-site with TLS 1.3 / AES-256-GCM [E05]; split tunnel [E06]; iptables allow/deny to 10.10.1.0/24 only [E11][E12]; two instances active [E16]
Implemented but limited	Two instances on one gateway with client-side selection [E21] [E22] (no high availability); iptables rules are network-based, not per user, and persistence after reboot was not verified; lab uses simulated LANs
Recommended, not implemented	Two-tier PKI, per-user certificates, TLS syslog, alerting, retention, multi-gateway HA, macOS testing

9 Limitations

- The LANs are simulated; there are no real LAN hosts behind either gateway.
- All machines share one Windows 10 host with 16 GB RAM; results may not reflect production hardware.
- One certificate and one account were used, so per-user separation is not shown.
- Hardening notes from client logs: persist-key is deprecated and auth-nocache is not set.
- Scalability and capacity figures in other documents are recommendations or estimates, not measurements.

Appendix A Evidence cited

ID	Content
E01	IPsec established, ESP captured, ping 0% loss
E05	Client connect, TLS 1.3, AES-256-GCM, pushed route
E06	Client routing table
E07	Server log: TOTP accepted, route pushed
E09	PAM configuration and auth.log
E10	Interfaces and key server.conf settings
E11 / E12	iptables FORWARD rules, VPN-DENY, client blocked
E16	Both OpenVPN instances active
E21 / E22	client-lb profile to 1194 / 1195
E27 / E28	rsyslog forwarding and received logs

Full configuration files stay on the gateways and are not reproduced here; the key settings are visible in the cited screenshots.